RUSTAMJI INSTITUTE OF TECHNOLOGY

BSF ACADEMY, GWALIOR (M.P.)

Major Project Report-I (CS706) Report

on

NetPhoenix : An Automated Self-Healing Network Failure Detection and Recovery



Submitted by

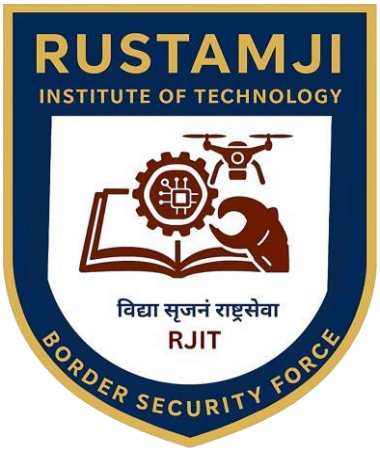
Malihah Rahaman	*Tripika Nama*	*Bhumi Shah*	*Unnati Patel*
(0902CS231056)	*(0902CS231124)*	*(0902IT231014)*	*(0902IT231057)*

Computer Science & Engineering 7th Semester
(2023-2027 batch)

Supervised by
Dr. Jagdish Makhijani, Registrar
Department of Computer Science & Engineering

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

RUSTAMJI INSTITUTE OF TECHNOLOGY, BSF ACADEMY,
TEKANPUR ,GWALIOR (M.P.)



CERTIFICATE

This is to certify that this project report on **" NetPhoenix : An Automated Self-Healing Network Failure Detection and Recovery "** is submitted by **"Bhumi Shah, Malihah Rahaman, Tripika Nama and Unnati Patel"** who carried out the project work under my supervision. I approve this project for submission of the Bachelor of Engineering in the Department of Computer Science & Information Technology, Rustamji Institute of Technology, BSF Academy, Tekanpur. Gwalior.

Dr. Jagdish Makhijani, Registrar

(Project Supervisor)

Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Aim and Objectives
5. Scope of the Project
6. Proposed System and Architecture
7. Technologies and Tools Used
8. Hardware and Software Requirements
9. Network Topology and IP Addressing
10. Implementation Details
11. Testing and Results
12. Troubleshooting and Lessons Learned
13. Work Completed
14. Work Remaining
15. Original Eight-Week Timeline and Current Progress
16. Conclusion
17. Reference

1.Abstract

NetPhoenix is our final-year major group project in the area of computer networks and network automation. The project studies how a network can stay available when a link fails, and how a Python-based monitoring layer can make failure detection and recovery verification more systematic. The approved design uses virtualized VyOS routers in EVE-NG, OSPF routing over redundant links, a Python automation component using Netmiko, and Wireshark for traffic and OSPF analysis.

This report covers the first stage of the project. We installed EVE-NG Community Edition inside VMware Workstation Pro and created a lab with two VyOS routers, R1 and R2, joined by two independent Ethernet links. We configured IPv4 addresses on both links using /30 subnets and enabled OSPF in Area 0. We inspected OSPF neighbor status and used ping to check connectivity. We then ran a manual failover experiment: R1 eth0 was disabled, connectivity over the alternate link was tested, and the interface was restored. We also enabled and saved the SSH service on both routers, installed Python 3.13.3 and Netmiko on Windows, installed Wireshark, and created a GitHub repository for screenshots and documentation.

A separate management connection, needed so that Python and Netmiko can reach the routers over SSH, is not yet working. R1 eth2 is configured with 192.168.135.200/25 and is up, but Windows ping to that address times out, and a packet capture on R1 showed an ARP request without a reply. This issue is unresolved. Because of it, SSH login from Windows and Netmiko integration have not yet been verified.

> **Status in one line.** OSPF currently performs the dynamic route selection. The Python automation is planned to monitor, detect, log and verify failures, but it has not been implemented or tested. The failover test was triggered manually by us, not by Python. Automated failure detection, automated recovery and Wireshark convergence analysis are pending, and no claim about them is made in this report.

2. Introduction

Modern networks are expected to remain available even when a cable, interface or device fails. A common way to improve availability is redundancy: placing more than one path between devices so that traffic can use another path when one fails. Redundant links alone are not enough, though. The routers must know about the alternate path, and administrators must notice the failure and confirm that service has recovered.

Dynamic routing protocols such as OSPF (Open Shortest Path First) help with the first part. OSPF routers exchange information about their links and calculate routes. When a link goes down, the routers update their routing tables and can use another available path. Detecting the failure, recording it, checking that connectivity really returned, and deciding whether any extra action is needed are usually done by an administrator or a monitoring tool.

NetPhoenix investigates this in a controlled virtual lab. We use VyOS routers in EVE-NG to build a small redundant network with OSPF. The planned Python layer, using the Netmiko library over SSH, will inspect router status, detect predefined failure conditions, log events and verify connectivity. Wireshark will be used to study OSPF behavior during failure and recovery.

This is Progress Report 1. It records what we have actually built and tested, what went wrong, and what remains.

3. Problem Statement

A link or router connectivity failure can interrupt communication when no usable alternate path exists. Even where redundant links exist, administrators still have to:

1 notice that a failure has happened;

2 find which link or device is affected;

3 check that the routing protocol has moved traffic to the alternate path; and 4 confirm that end-to-end connectivity has recovered.

Doing this by hand is slow and inconsistent, especially as networks grow. Our project asks how dynamic routing (OSPF) and a Python-based monitoring layer can make these tasks more systematic in a controlled virtual lab, without needing any physical networking hardware.

4. Aim and Objectives

4.1 Aim

To design and demonstrate a virtual self-healing network prototype that combines redundant routing with Python-based network automation for failure detection, recovery verification and traffic analysis.

4.2 Objectives and Current Status

No.	Objective	Status at 18 Oct 2026
1	Build a virtual network using EVE-NG and VyOS routers.	Completed
2	Configure IPv4 addressing and OSPF routing across redundant links.	Completed; neighbor status inspected
3	Simulate a link failure and test connectivity through the alternate path.	Performed manually
4	Enable SSH and develop Python monitoring using Netmiko.	SSH service configured on both routers; Python and Netmiko installed; remote login and Netmiko connection not yet verified
5	Detect predefined failures and record the events.	Pending
6	Verify OSPF convergence and connectivity after failure; perform controlled recovery actions where required.	Pending
7	Use Wireshark to examine traffic and OSPF behavior during failure and recovery.	Pending (Wireshark installed)
8	Document configuration, troubleshooting, test results and limitations.	In progress

5. Scope of the Project

5.1 Within scope

- A virtual lab built with EVE-NG and VyOS; no physical hardware.
- A two-router topology with two redundant point-to-point links.
- OSPF (Area 0) for dynamic routing.
- SSH-based management of the routers from a Python script using Netmiko.
- Predefined failure conditions, such as a link going down or a router interface becoming unreachable.
- Logging of detected failures and verification of connectivity after recovery.
- Packet capture and OSPF behavior analysis with Wireshark.

5.2 Outside the current scope

- Production networks or physical devices.
- Large topologies or multi-area OSPF designs.
- Detection of arbitrary or unknown failures. The system is meant to detect predefined conditions only.
- Claims about performance improvement or measured convergence time. These can be made only after testing and measurement have been carried out.

OSPF currently handles dynamic route selection. The planned Python system will monitor, detect, log and verify. It is not intended to replace OSPF.

6. Proposed System and Architecture

6.1 Three-layer design

Network layer (implemented in this stage). VyOS routers R1 and R2, two redundant Ethernet links, IPv4 addressing and OSPF Area 0. This layer calculates routes and can use the remaining path when a link is unavailable, provided the remaining interfaces and routing are operational.

Automation layer (planned). A Python program using Netmiko over SSH will log in to the routers, read interface and routing status, check for predefined failure conditions, record events with timestamps, verify connectivity and, where necessary, perform controlled recovery actions.

Analysis layer (planned). Wireshark will capture traffic before, during and after a failure so that we can examine OSPF messages and convergence behavior.

6.2 Intended workflow

Step	Stage	Status
1	Virtual network in EVE-NG with VyOS routers and redundant links	Implemented
2	OSPF routing in Area 0	Implemented
3	Failure introduced on one link	Done manually so far
4	Python + Netmiko monitoring over SSH	Planned
5	Detect and log the failure	Planned
6	Verify alternate-path connectivity and OSPF convergence	Planned (manual ping check done)
7	Wireshark analysis of OSPF behavior	Planned

6.3 Management network

Netmiko needs SSH access to the routers. We are setting up a separate management path, independent of the OSPF links. It connects router eth2 to a network object named Mgmt (Cloud0) in EVE-NG, which connects to the Windows host through the VMware VMnet8 adapter. This path is not working yet (see Sections 11.7 and 13).

7. Technologies and Tools Used

Technology / tool	Purpose in the project	Status
EVE-NG Community Edition 6.2.0-4	Builds and runs the virtual network topology	In use
VMware Workstation Pro 17.6.4	Hosts the EVE-NG virtual machine	In use
VyOS (vyos-2026.10.17-0025-rollin g-generic-amd64)	Virtual routers, interface and routing configuration	In use
IPv4 and subnetting	Addressing for the two router-to-router links (/30 subnets)	In use
OSPF	Dynamic routing and alternate-path selection	Configured
Linux / VyOS command line	Configuration, verification and troubleshooting	In use
SSH / OpenSSH	Remote management of the routers	Service enabled on both routers; remote login from Windows not verified
tcpdump	Inspecting ARP packets during management-network troubleshooting	Used
Python 3.13.3	Planned monitoring and automation scripts	Installed on Windows
Netmiko	Planned Python-based router management over SSH	Installed with pip; not yet tested against the routers
Wireshark	Packet capture and OSPF analysis	Installed; analysis pending
EVE-NG console client	Opens router consoles from the EVE-NG topology	Installed and working
Git and GitHub	Version control, screenshots, documentation	Repository created

8. Hardware and Software Requirements

8.1 Requirements as planned

- Laptop or PC with at least 8 GB RAM.
- At least 60 GB of free storage recommended.
- Hardware virtualization support.
- No physical networking hardware is required.
- Software: EVE-NG Community Edition, VyOS, VMware Workstation Pro, Ubuntu Server, Python 3, Netmiko, OpenSSH, Wireshark, Git/GitHub.

8.2 Actual development environment

Item	Details
Host operating system	Windows 11 Home
Laptop	Acer Nitro 5
Processor	Intel Core i5-13420H
RAM	16 GB
Graphics	NVIDIA RTX 4050
Virtualization software	VMware Workstation Pro 17.6.4
Network emulator	EVE-NG Community Edition 6.2.0-4
Router image	vyos-2026.10.17-0025-rolling-generic-amd64.iso
Python	3.13.3 (Windows)
Netmiko	Installed through pip (Windows)
Packet analyzer	Wireshark (installed)
Console access	EVE-NG console client (Windows)
Repository	GitHub repository "NetPhoenix"

Our machine has more RAM than the stated minimum, which is enough to run the host tools, VMware and the two-router lab together.

9. Network Topology and IP Addressing

9.1 Topology

The EVE-NG lab contains two VyOS routers, R1 and R2, joined by two independent Ethernet links (eth0 and eth1 on each router). The second link provides redundancy: if one link is disabled, the other can carry traffic, provided its interfaces and routing are working. The

topology also shows a Mgmt (Cloud0) object attached to eth2 on the routers, used for the separate management connection described in Section 11.7.

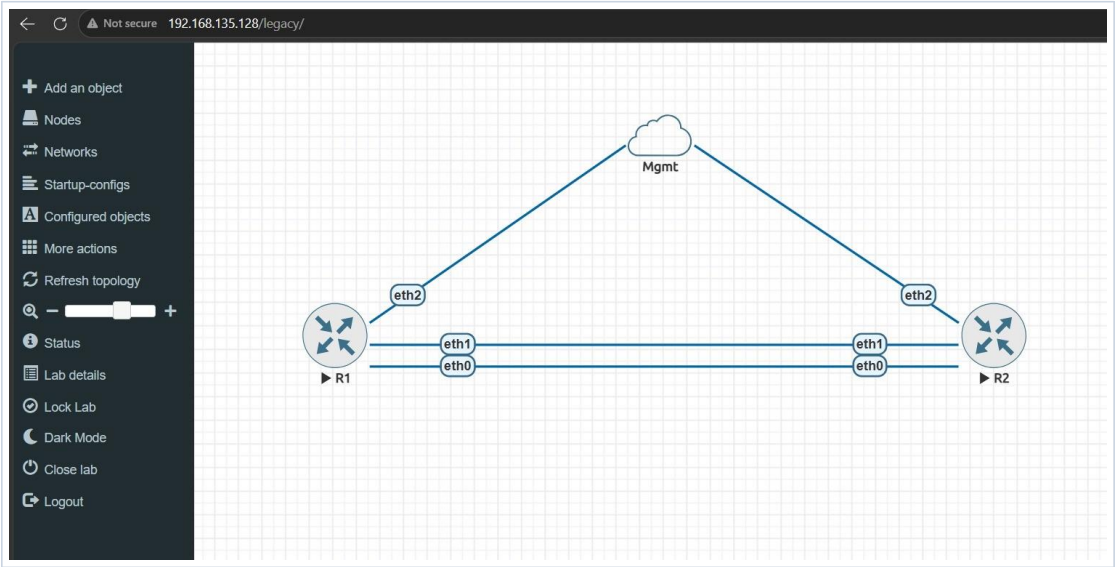


Figure 1. Virtual network topology in EVE-NG showing R1 and R2 with two links (screenshots/net-topology.png).

9.2 IP addressing of the OSPF links

Link	Router	Interface	IPv4 address	Subnet
Link 1	R1	eth0	10.0.12.1/30	10.0.12.0/30
Link 1	R2	eth0	10.0.12.2/30	10.0.12.0/30
Link 2	R1	eth1	10.0.13.1/30	10.0.13.0/30
Link 2	R2	eth1	10.0.13.2/30	10.0.13.0/30

A /30 subnet has two usable host addresses, one for each end of a point-to-point link, so no addresses are wasted.

9.3 Management addressing (separate from the OSPF links)

Item	Address	Note
Windows VMware Network Adapter VMnet8	192.168.135.1/24	Windows side
EVE-NG management interface pnet0	192.168.135.128/24	Windows can reach this address
R1 eth2 (to Mgmt / Cloud0)	192.168.135.200/25	Configured and up; Windows cannot reach it yet

R1 eth2 is not part of the two-link OSPF topology. The OSPF links eth0 and eth1 were kept separate from the management troubleshooting and left unchanged.

10. Implementation Details

10.1 EVE-NG and VyOS setup

We installed VMware Workstation Pro 17.6.4 on the Windows 11 laptop and ran EVE-NG Community Edition 6.2.0-4 as a virtual machine. The EVE-NG web interface is opened from Windows using the EVE-NG management address (192.168.135.128). In EVE-NG we created a lab and added two VyOS nodes, R1 and R2, using the VyOS rolling image vyos-2026.10.17-0025-rolling-generic-amd64.iso. Both routers were installed and booted successfully. We installed the EVE-NG console client on Windows so that the router terminals could be opened directly from the topology page.

10.2 Interface IP configuration

Each router-to-router link was given its own /30 subnet. The interface addresses were set in VyOS configuration mode. For R1 the commands follow this pattern (R2 uses the .2 addresses from Section 10.2):

```
configure set interfaces ethernet eth0 address 10.0.12.1/30 set interfaces ethernet eth1 address 10.0.13.1/30 commit save
```

10.3 OSPF configuration

OSPF was configured on both routers in Area 0. The network statements include both link subnets, 10.0.12.0/30 and 10.0.13.0/30. OSPF neighbor status was inspected from the VyOS CLI with run show ip ospf neighbor (the shortened form run sh ip ospf neigh also works and appears in our screenshots).

```
set protocols ospf area 0 network 10.0.12.0/30 set protocols ospf area 0 network 10.0.13.0/30 commit save
run show ip ospf neighbor
```

Figure 2 was captured on R2 right after the Link 2 address and OSPF network statement were entered, so the second adjacency was still forming. It shows router ID 10.0.13.1 (R1) as the neighbor on both interfaces. On eth0 the neighbor was in the Full/Backup state. On eth1 it was in the 2-Way/DROther state with an up time of about 26 seconds. We therefore do not claim a Full adjacency on every interface in this capture. A later capture (Figure 5) is discussed in Section 11.6.

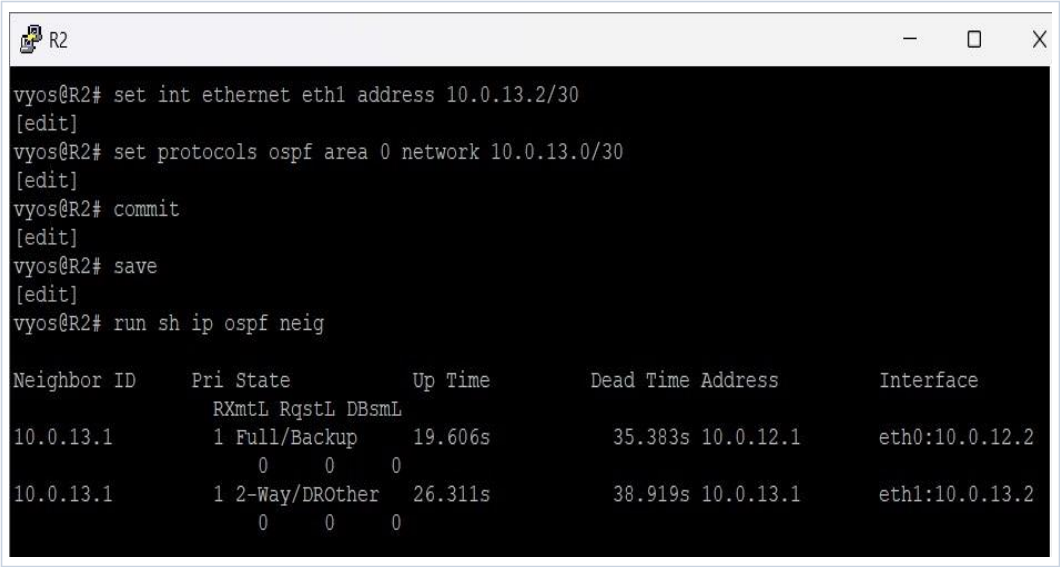


Figure 2. OSPF neighbor status before the failure test (screenshots/ospf-neighbors-before-failover.png).

10.4 SSH service configuration on both routers

The SSH service was enabled on **both R1 and R2** with the VyOS command below. The configuration was committed and saved on each router.

```
set service ssh port 22
commit save
```

Configured vs verified. SSH service *configuration* has been performed on both routers. However,
Windows-to-router management connectivity is not yet working reliably, so a successful *remote SSH login from Windows* and Netmiko integration have **not** been verified. These are separate things, and we do not claim the second one.

10.5 Connectivity tests

Before introducing any failure, we used ping to confirm that the routers could communicate. Figure

shows a ping from R2 to 10.0.13.1 (R1's address on Link 2): 4 packets transmitted, 4 received, 0% packet loss, with round-trip times between 0.567 ms and 2.543 ms (average 1.518 ms).

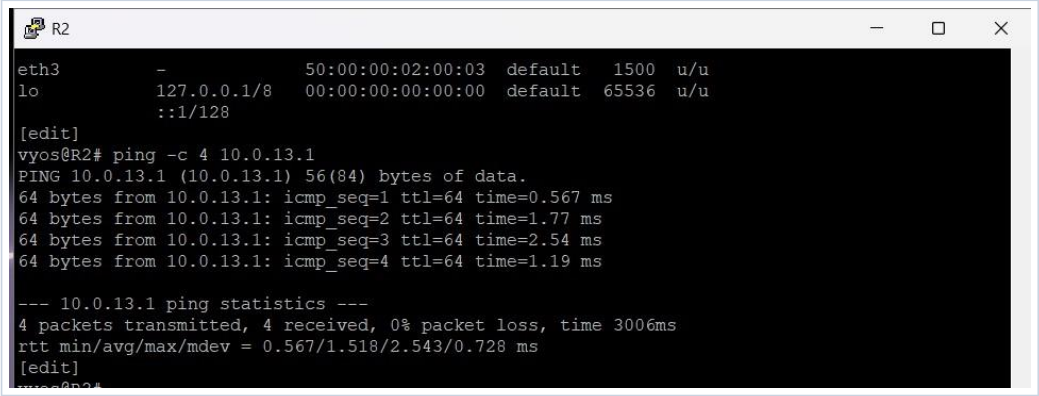


Figure 3. Connectivity test before simulating the link failure (screenshots/ping-before-failover.png).

10.6 Manual failover experiment

To simulate a link failure we disabled R1 eth0 (Link 1) from VyOS configuration mode, tested connectivity over the remaining link, and then restored the interface. The failure was triggered by us at the router console. **Python did not detect or trigger it.**

Commands used to create and clear the failure:

```
# on R1: simulate the failure set
interfaces ethernet eth0 disable commit

# on R1: restore the link afterwards delete
interfaces ethernet eth0 disable commit
save
```

Procedure followed:

1. Confirm connectivity between the routers with ping (Figure 3).
2. Disable R1 eth0 and commit.
3. Test connectivity over the alternate link while eth0 is disabled (Figure 4).
4. Restore R1 eth0, commit and save.
5. Check OSPF neighbor status again (Figure 5).

In Figure 4, a first attempt using run ping -c 4 10.0.13.1 returned "Unknown host: -c", because the VyOS run wrapper did not accept that option form (see Section 13.4). Running run ping 10.0.13.1 worked: 11 packets transmitted, 11 received, 0% packet loss, round-trip times between 0.600 ms and 2.702 ms (average 1.238 ms).

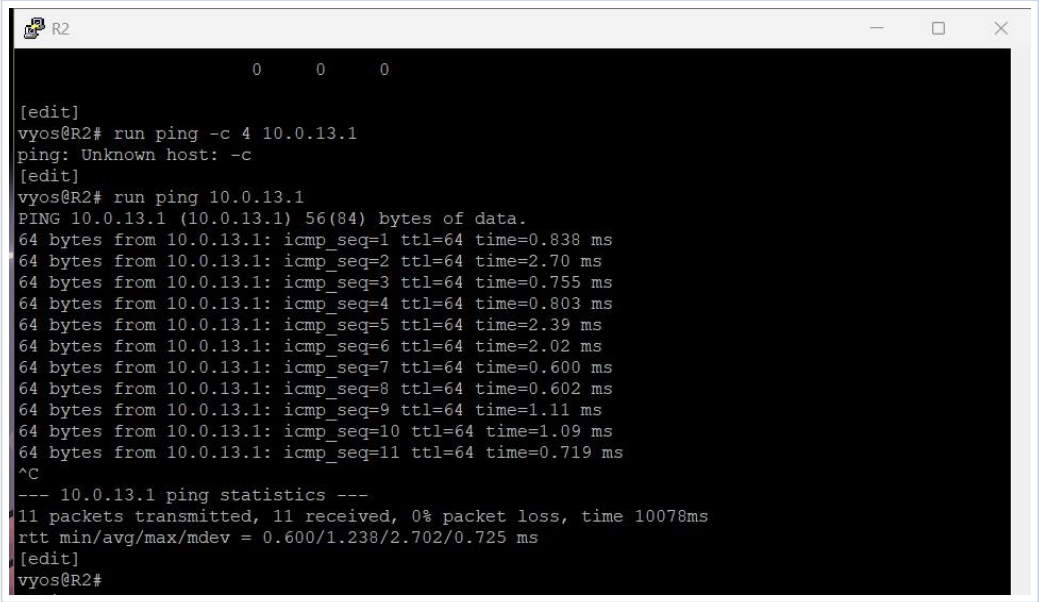


Figure 4. Ping test during the manual failover experiment (screenshots/ping-during-failover.png).

After restoring the interface we checked OSPF again. In the capture in Figure 5, R2 shows router

10.0.13.1 as Full/Backup on eth0 (up time 4 minutes 44 seconds) and as Full/DR on eth1 (up time 12 minutes 18 seconds). Both interfaces were in the Full state at that point. The shorter up time on eth0 is consistent with that adjacency having been re-established after the interface was restored, although we have not measured this precisely.

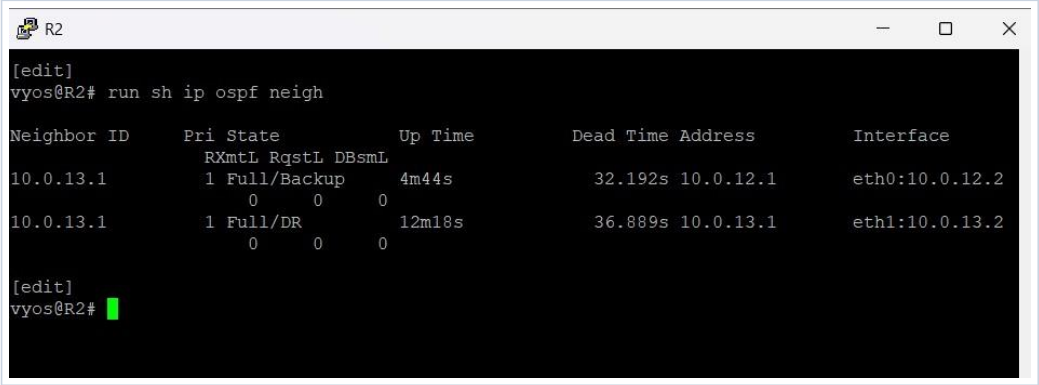


Figure 5. OSPF status after the failover experiment and restoration (screenshots/ospf-neighbors-after-failover.png).

What this experiment shows, and what it does not. It shows that the second link stayed usable while the first link was disabled, and that OSPF neighbors were Full on both interfaces after restoration. It is a basic redundant-link check. The ping target (10.0.13.1) is a directly connected address on Link 2, so the test confirms that link, not a measured OSPF re-routing event. We did not record convergence time, and the screenshots do not themselves show the disable command; that step is documented from our procedure. This is **not** a demonstration of an automated self-healing system.

10.7 Management interface configuration and current issue

To let Python and Netmiko reach the routers over SSH, we started building a separate management connection. The environment and the work done so far are summarized below.

Item	Detail
Windows adapter	VMware Network Adapter VMnet8: 192.168.135.1/24
EVE-NG management	pnet0: 192.168.135.128/24, bridged to the EVE management interface eth0 with virtual interfaces attached
EVE-NG topology object	Network object "Mgmt" (Cloud0) added to the lab; R1 eth2 connected to it
R1 eth2 configuration	192.168.135.200/25; interface confirmed UP; address visible in run show interfaces ethernet eth2
Routing table on R1	Connected route 192.168.135.128/25 dev eth2 present
R1 eth2 MAC address	50:00:00:01:00:02
Bridge checks on EVE-NG	brctl show and brctl showmacs pnet0 were checked
Windows to EVE-NG	Windows can reach 192.168.135.128
Windows to R1 eth2	Ping to 192.168.135.200 times out; Windows ARP table shows no resolved MAC for that address
Capture on R1 eth2	tcpdump showed an ARP request for 192.168.135.200 from 192.168.135.1, but no ARP reply in that capture

The interface is up and has an address, yet management connectivity remains unresolved. The next step is to troubleshoot the Cloud0 / Layer 2 management path, then verify SSH from Windows, and finally test Netmiko. The topology also shows R2 eth2 attached to the Mgmt object; our troubleshooting so far has focused on R1, and management access to R2 has not been verified.

The OSPF links eth0 and eth1 were not changed during this work.

Status: unresolved. We do not claim that the management connection works.

11. Testing and Results

The table below lists the tests we have actually carried out and what we observed. No measured convergence times or performance figures are reported, because those measurements have not been made.

Test	What we did	Observed result	Evidence
Router boot	Started R1 and R2 in EVE-NG	Both routers booted and are accessible from the console client	Figure 1
Link addressing	Configured /30 addresses on eth0 and eth1	Addresses configured on both links	Section 10.2
OSPF neighbors (initial)	run sh ip ospf neigh on R2 after adding Link 2	eth0: Full/Backup; eth1: 2-Way/DROther (adjacency still forming)	Figure 2
Ping before failure	Ping from R2 to 10.0.13.1	4 sent, 4 received, 0% loss	Figure 3
Manual link failure	Disabled R1 eth0	Alternate link tested while eth0 disabled	Section 11.6
Test	**What we did**	**Observed result**	**Evidence**
Ping during failure	Ping from R2 to 10.0.13.1	11 sent, 11 received, 0% loss	Figure 4
Restoration and OSPF check	Deleted the disable setting on R1 eth0; checked neighbors	eth0: Full/Backup; eth1: Full/DR	Figure 5
SSH service	set service ssh port 22 on R1 and R2, commit and save	Service configured; remote login from Windows not verified	Section 11.4
Windows to R1 management IP	Ping 192.168.135.200 from Windows	Times out; ARP request seen on R1 with no reply	Section 11.7

11.1 Summary of status

Component	Status	Note
EVE-NG and VyOS lab	Completed	Two routers running in the virtual lab
Dual Ethernet links and IPv4	Completed	Two /30 link subnets configured
OSPF configuration	Implemented	Neighbor status checked from the VyOS CLI
Manual failover experiment	Performed	Alternate-link connectivity observed during the manual test
SSH service on R1 and R2	Configured	Enabled and saved; remote login not verified
Windows management access	In progress	ARP / Cloud0 issue remains
Python + Netmiko failure detection	Pending	Requires reliable SSH management access
Automated recovery and logging	Pending	Not implemented or tested
Wireshark convergence analysis	Pending	Captures and analysis remain to be done

12. Troubleshooting and Lessons Learned

This section records the issues we met and what we checked. It is factual and does not claim that the management-network issue has been solved.

12.1 OSPF neighbor, router ID, interface and network-statement checks

While building the lab we checked the interface addresses on both routers, the OSPF network statements, the router IDs and the neighbor states. The neighbor output (Figures 2 and 5) showed the neighbor ID 10.0.13.1 for R1. We also saw that a newly added link can show a transitional state (2-Way) before the adjacency becomes Full, so the neighbor table should be re-checked after a short wait. **Lesson:** verify addresses, network statements and neighbor state before relying on dynamic routing.

12.2 Ping tests to verify router connectivity

Ping was used before, during and after the failure experiment. **Lesson:** testing at all three points helps show whether a redundant path works only under normal conditions or also when a link is down.

12.3 Manual interface disable and restoration

The failure was created with set interfaces ethernet eth0 disable and cleared with delete interfaces ethernet eth0 disable, each followed by commit (and save after restoration).

Lesson: VyOS changes take effect only on commit and persist only after save, which matters for the later automated tests.

12.4 VyOS configuration mode vs operational mode

Some commands failed because they were entered in the wrong mode, or were Linux-style commands that the VyOS run wrapper did not accept. An example is visible in Figure 4, where run ping -c 4 10.0.13.1 returned "Unknown host: -c" and the plain run ping 10.0.13.1 worked. We checked which mode we were in and used the supported operational commands for verification. **Lesson:** configuration and operational commands have different purposes and syntax, and our Netmiko script will need to account for the mode it is in.

12.5 EVE-NG console client setup

Opening router consoles from the EVE-NG topology required a console client to be set up on Windows. We installed and configured it, and the VyOS terminals now open from the topology page.

12.6 SSH service configuration on both routers

We enabled the SSH service on R1 and R2 using set service ssh port 22, followed by commit and save. This completes the router side of the SSH service configuration. It does not by itself prove that Windows can log in, because that also needs a working network path.

12.7 Unresolved management network / Cloud0 issue

R1 eth2 is connected to the Mgmt (Cloud0) object and configured with 192.168.135.200/25. The interface is UP, the address is visible in the interface output, and the routing table has the connected route 192.168.135.128/25 on eth2. Windows (192.168.135.1/24 on VMnet8) can reach EVE-NG at 192.168.135.128, but ping to 192.168.135.200 times out and the Windows ARP table has no resolved MAC address for it. A tcpdump capture on R1 eth2 showed an ARP request asking for 192.168.135.200 from 192.168.135.1, with no ARP reply in that capture. We also checked the EVE-NG bridges with brctl show and brctl showmacs pnet0. The cause is not yet identified.

Status: unresolved.

Checks we plan next in the Cloud0 / Layer 2 path: confirm the Cloud0 / pnet0 attachment in EVE-NG, compare what the bridge learns with what R1 sends, and review the subnet-mask settings used on R1 eth2 (/25) and on the Windows VMnet8 and pnet0 side (/24).

12.8 Why reliable management connectivity is needed before Netmiko

Netmiko works by opening an SSH session from Python to the router and sending CLI commands. If the Windows host cannot reach the router's management address, no SSH session can be established, so a script cannot read interface state, detect a failure or log results. Trying to test detection logic on an unreliable management path would make it impossible to tell script errors from network errors. We are therefore fixing the management path first. For the same reason we left the working OSPF links unchanged while troubleshooting.

13. Work Completed

- Installed and configured EVE-NG Community Edition 6.2.0-4 in VMware Workstation Pro 17.6.4.
- Installed and started two VyOS routers, R1 and R2.
- Built a redundant two-link topology.
- Configured IPv4 addresses on both router-to-router links.
- Configured OSPF Area 0 on both routers and checked OSPF neighbor status.
- Verified basic ping connectivity before the failure experiment.
- Performed a manual link-failure experiment by disabling R1 eth0.
- Verified ping connectivity through the alternate link during the experiment.
- Restored R1 eth0 and checked OSPF status again.
- Enabled and saved the SSH service configuration on both R1 and R2.
- Installed Python 3.13.3 and Netmiko on Windows, and installed Wireshark.
- Installed the EVE-NG console client on Windows.
- Created the NetPhoenix GitHub repository and uploaded the screenshots.
- Started management-network troubleshooting for the Netmiko integration (unresolved).

14. Work Remaining

1. Resolve the management network / Cloud0 connectivity issue.
2. Verify SSH login from Windows to R1 and R2.
3. Use Netmiko to connect to both routers.
4. Write Python code to read interface and routing status.
5. Implement monitoring of predefined network failure conditions.
6. Detect a simulated link or connectivity failure.
7. Record the detected failure and the test result.
8. Verify OSPF convergence and connectivity through the alternate path.
9. Implement controlled recovery actions where necessary, then test them safely.
10. Capture traffic in Wireshark before, during and after failures.
11. Analyze OSPF messages and convergence behavior.
12. Repeat the tests and record observations.
13. Evaluate the prototype and document its limitations.
14. Update the final report, GitHub repository and final demonstration.

15. Original Eight-Week Timeline and Current Progress

Period	Planned activities (original plan)	Current progress	Status
Weeks 1-2	EVE-NG, VyOS, network topology and OSPF configuration	Lab, topology, addressing, OSPF and a manual failover test completed	Done
Weeks 3-4	Ubuntu, SSH, Python and Netmiko	SSH enabled on both routers; Python and Netmiko installed on Windows; management connection to the routers unresolved; Ubuntu not yet used	In progress
Period	**Planned activities (original plan)**	**Current progress**	**Status**
Weeks 5-6	Network failure detection, automated recovery and connectivity verification	Not started	Pending
Weeks 7-8	Failure testing, Wireshark analysis, performance evaluation, documentation and final demonstration	Wireshark installed; GitHub documentation started; analysis and evaluation not started	Pending

The management-connectivity issue is the main item holding back the Weeks 3-4 activities, and the later stages depend on it. Our priority is to clear it so that the remaining stages can proceed.

16.Conclusion

The first stage of NetPhoenix has established the virtual networking foundation for the major project. Two VyOS routers are connected through redundant links, OSPF is configured, and an initial manual test showed that the alternate link stayed usable while the first link was disabled. After the interface was restored, OSPF neighbors were in the Full state on both links. SSH has been enabled on both routers, and Python, Netmiko and Wireshark are installed.

The complete self-healing workflow has not been implemented. OSPF currently handles dynamic route selection; the Python component that will monitor, detect, log and verify failures is still planned. The immediate priority is to resolve the management connectivity issue and verify SSH access from Windows, so that we can begin integrating Netmiko. After that we will implement failure monitoring, verify recovery behavior, and use Wireshark to study OSPF during failures. Results from those stages will be added to the final project documentation.

17. References

1 VyOS Documentation. https://docs.vyos.io/

2 EVE-NG. https://www.eve-ng.net/

3 Netmiko project (K. Byers). https://github.com/ktbyers/netmiko

4 Wireshark Documentation. https://www.wireshark.org/docs/

5 J. Moy, "OSPF Version 2", RFC 2328, IETF, 1998. https://www.rfc-editor.org/rfc/rfc2328

6 NetPhoenix project repository (code and screenshots). https://github.com/gulpchronics/NetPhoenix